DATE : 01.08.2025

DT/NT : DT

LESSON : DEEP LEARNING

SUBJECT : TRANSFER LEARNING

BATCH : B287

DATA SCIENCE

TECHPRO EDUCATION

techproeducation.com

+1 (585) 304 29 59

TRANSFER LEARNING

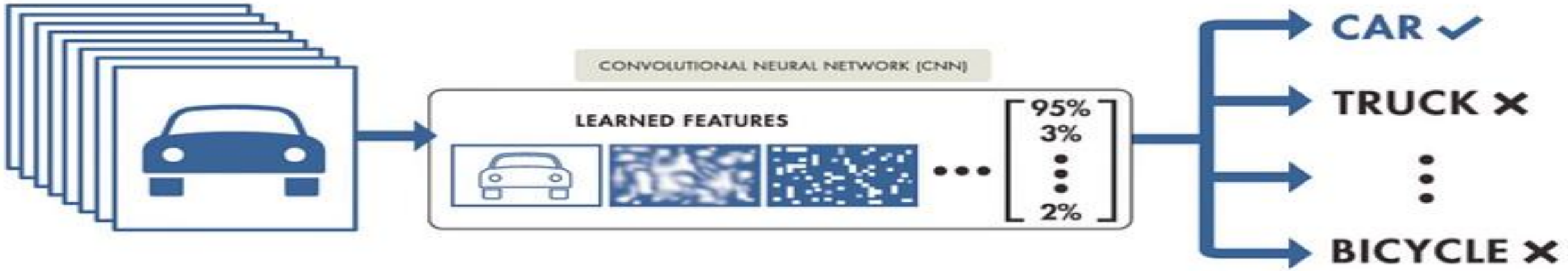
What is Transfer Learning?

https://www.youtube.com/watch?v=BqqfQnyjmgg

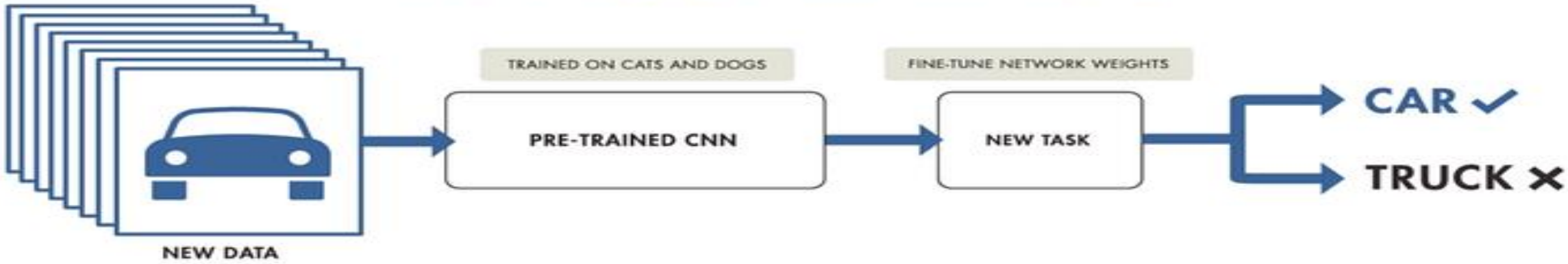
First 1.40 period

TRANSFER LEARNING

TRAINING FROM SCRATCH



TRANSFER LEARNING



TRANSFER LEARNING

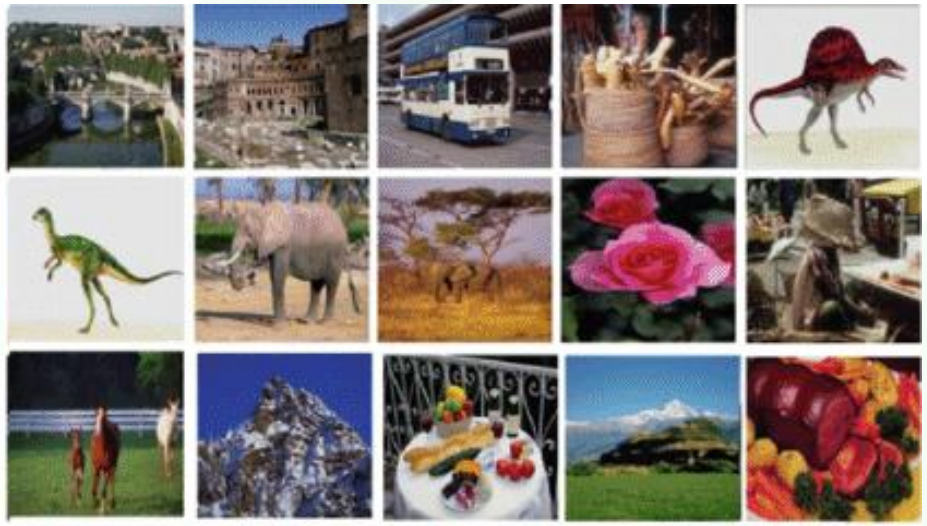
ImageNet

1,000 object classes (categories).
Images:
- 1.2 M train
- 100k test.

TRANSFER LEARNING

ImageNet

(a) ImageNet Synset: One sample image from each category



(b) Corel-1000 Dataset: Sample images from each category

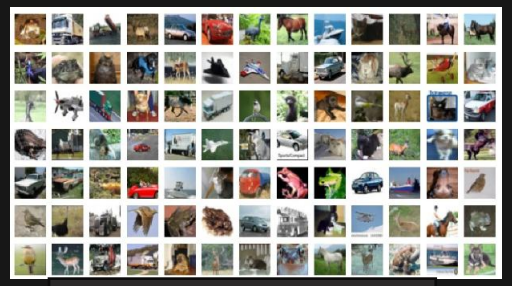
(c) Caltech-256 Dataset: One sample image from each category

(d) Caltech-101 Dataset: One sample image from each category

ImageNet

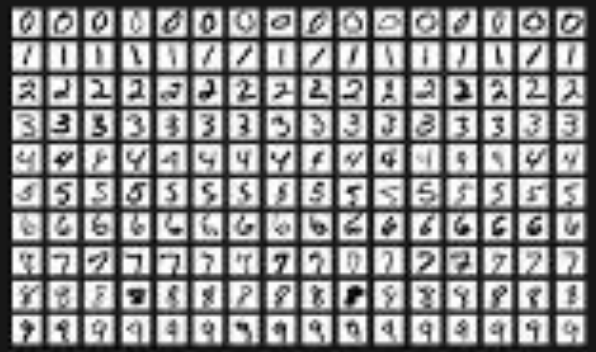
ImageNet

IMAGENET



CIFAR 100

CIFAR 10



MNIST

Derin Öğrenme Modellerinin Eğitiminde Kullanılan 5 Önemli Veri Seti

Muslum Yildiz · Following

Published in Academy Team · 9 min read · Apr 28, 2023

https://medium.com/academy-team/derin-%C3%B6%C4%9Frenme-modellerinin-e%C4%9Fitiminde-kullan%C4%B1lan-5-%C3%B6nemli-veri-seti-788a1d71f9fd

COCO

The **COCO (Common Objects in Context)** dataset is used for **visual recognition** tasks such as **object detection**, **object classification**, and **semantic segmentation**. The COCO dataset was **created** by the **Microsoft COCO** team and contained **80 different object classes**.

These object classes include various objects such as people, animals, furniture, food, and electronic devices.
The COCO dataset consists of more than 1.2 million images, various semantic features of objects in these images and their labelled sections.

COCO dataset trains machine learning algorithms for **object detection** and other **visual recognition** tasks.

The **YOLO (You Only Look Once)** model was **trained using the COCO** dataset.

COCO

The **YOLO model can** also **be trained** using the **ImageNet** dataset. The ImageNet dataset contains approximately 1 million different object classes. **However**, the **object classes** in the **ImageNet dataset** may be **more diverse** and specific than those in the COCO dataset. This may increase the difficulty of visual recognition tasks such as object detection, classification, and segmentation.

In addition, the **YOLO model can be trained with other datasets** besides the object classes in the ImageNet dataset. However, the **COCO dataset** stands out with its performance in tasks such as **object detection** and **classification**, and therefore, the **YOLO model is usually trained with it**. The COCO dataset contains more specific and diverse object classes for object detection, which helps the YOLO model produce better results.

ImageNet vs. COCO

- While I**mageNet** is typically used to train **image classification** models, **COCO** is used for a broader problem of **object detection and segmentation (especially in YOLO, CV)**.

- While **ImageNet** offers **a high level of diversity** across classes, **COCO includes more specific elements** to **capture** the **complexity** and **diversity** of results.

ImageNet

COCO

R-CNN and YOLO

R-CNN and YOLO are two deep learning models used for visual recognition tasks such as object detection and classification. Although they are fundamentally designed for similar purposes, they differ in some ways.

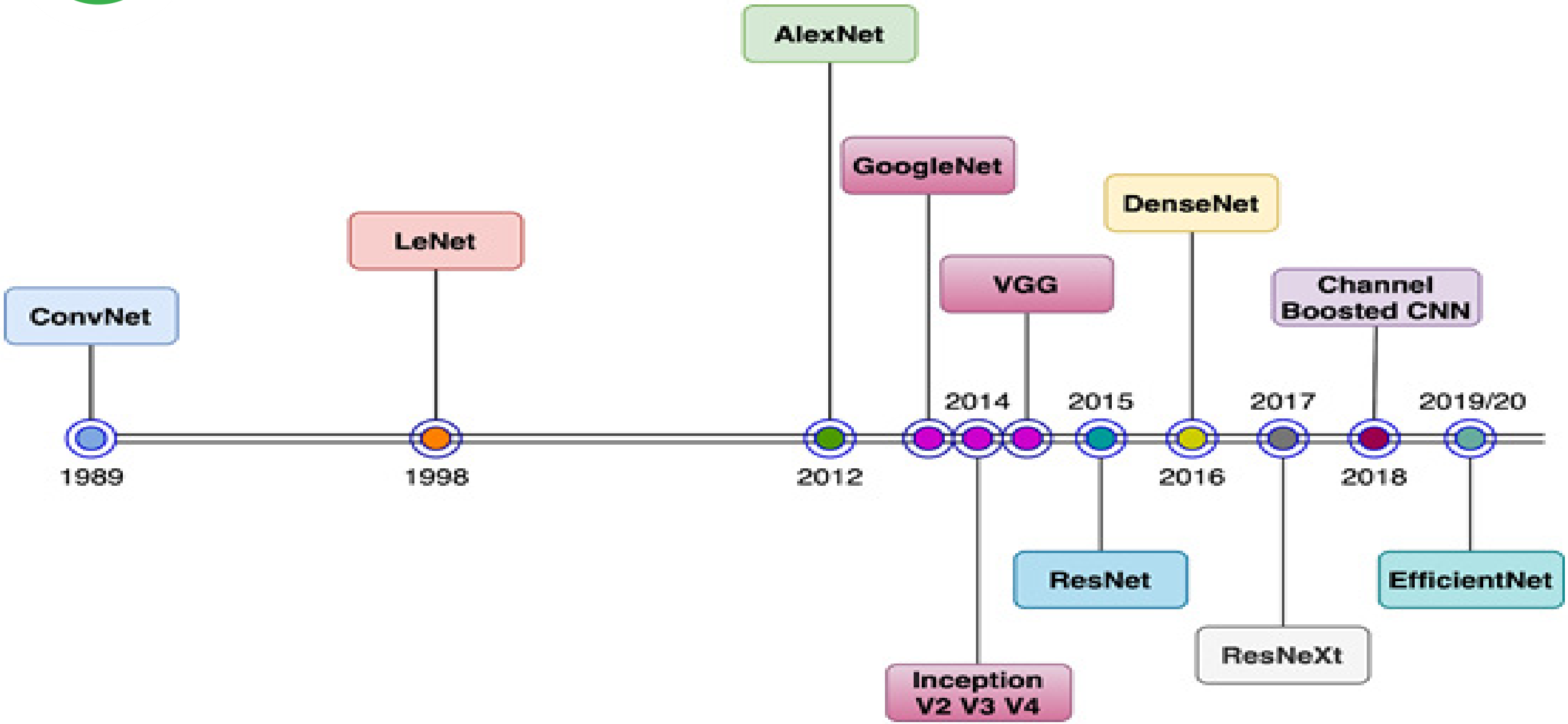
R-CNN (Region-based Convolutional Neural Network) first determines regional proposals in images for object detection and then passes these proposals through separate feature extraction stages to perform object detection. R-CNN can also perform the segmentation task of objects.

On the other hand, YOLO (You Only Look Once) is a deep learning model developed to provide faster results in object details and descriptions. YOLO separates and displays the object on the screen by processing it at once. YOLO can also be used for segmentation, but it gives weaker results.

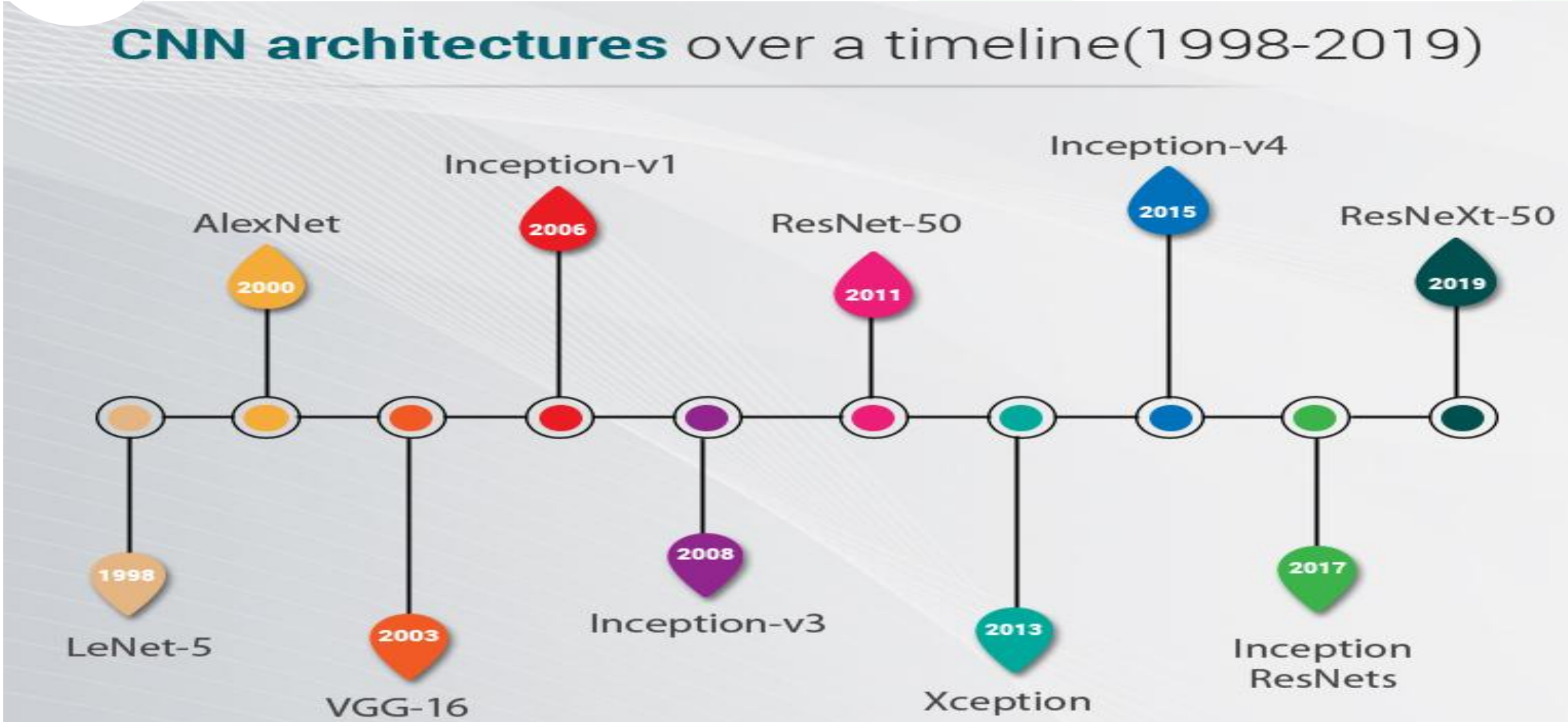
In conclusion, although R-CNN and YOLO are designed for similar purposes, they offer different approaches and different performance characteristics**. R-CNN provides better results in object detection and segmentation tasks**, while **YOLO offers faster results and better real-time performance**.

We will learn/use YOLO at computer vision section.

TRANSFER LEARNING



TRANSFER LEARNING



TRANSFER LEARNING

Keras

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7
NASNetLarge	343	82.5%	96.0%	88.9M	533	344.5	20.0
EfficientNetB0	29	77.1%	93.3%	5.3M	132	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	186	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	186	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	210	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	258	308.3	15.1
EfficientNetB5	118	83.6%	96.7%	30.6M	312	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	360	958.1	40.4
EfficientNetB7	256	84.3%	97.0%	66.7M	438	1578.9	61.6
EfficientNetV2B0	29	78.7%	94.3%	7.2M	-	-	-
EfficientNetV2B1	34	79.8%	95.0%	8.2M	-	-	-
EfficientNetV2B2	42	80.5%	95.1%	10.2M	-	-	-
EfficientNetV2B3	59	82.0%	95.8%	14.5M	-	-	-
EfficientNetV2S	88	83.9%	96.7%	21.6M	-	-	-
EfficientNetV2M	220	85.3%	97.4%	54.4M	-	-	-
EfficientNetV2L	479	85.7%	97.5%	119.0M	-	-	-
ConvNeXtTiny	109.42	81.3%	-	28.6M	-	-	-
ConvNeXtSmall	192.29	82.3%	-	50.2M	-	-	-
ConvNeXtBase	338.58	85.3%	-	88.5M	-	-	-
ConvNeXtLarge	755.07	86.3%	-	197.7M	-	-	-
ConvNeXtXLarge	1310	86.7%	-	350.1M	-	-	-

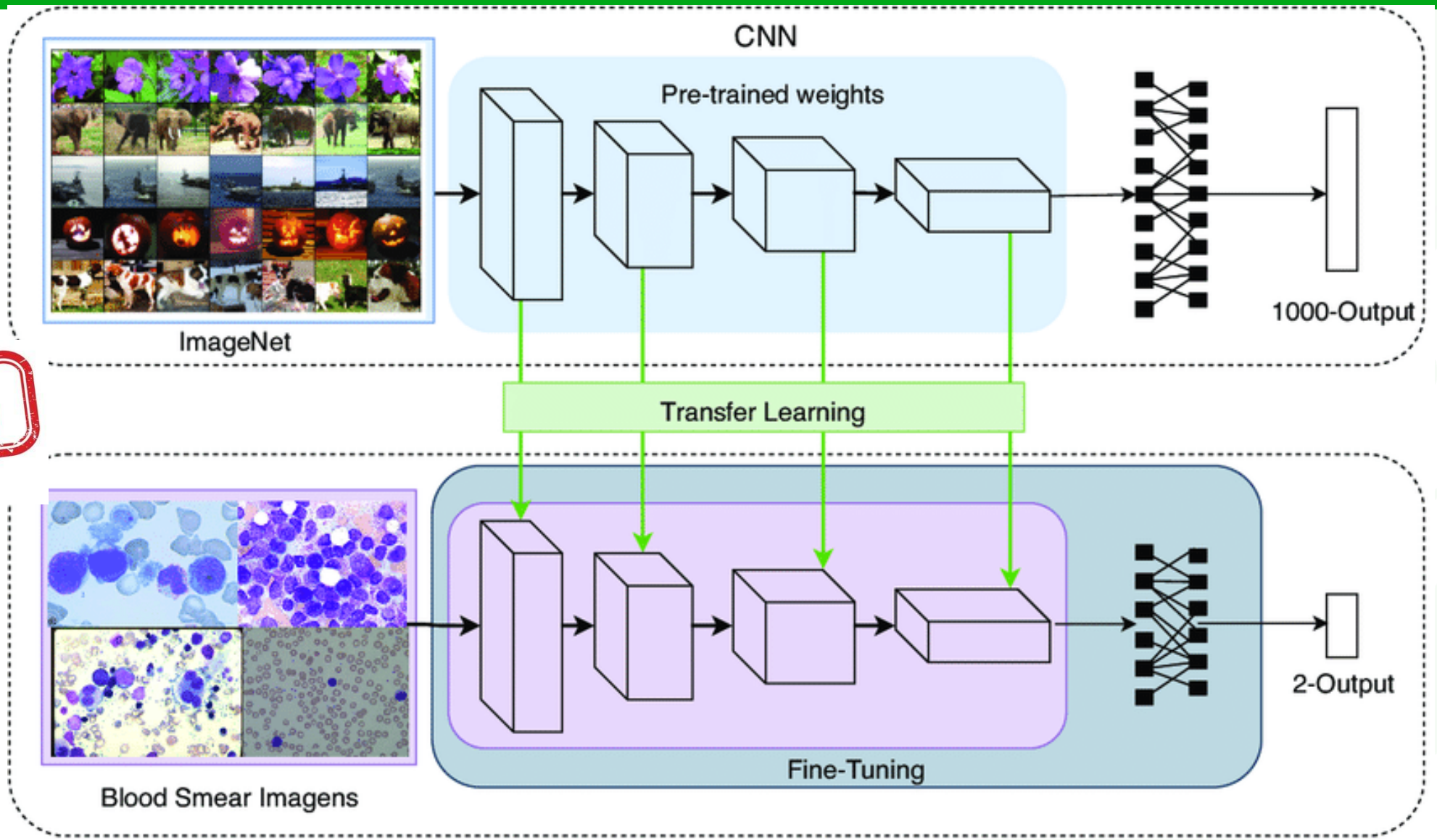
Keras Applications

https://keras.io/api/applications/

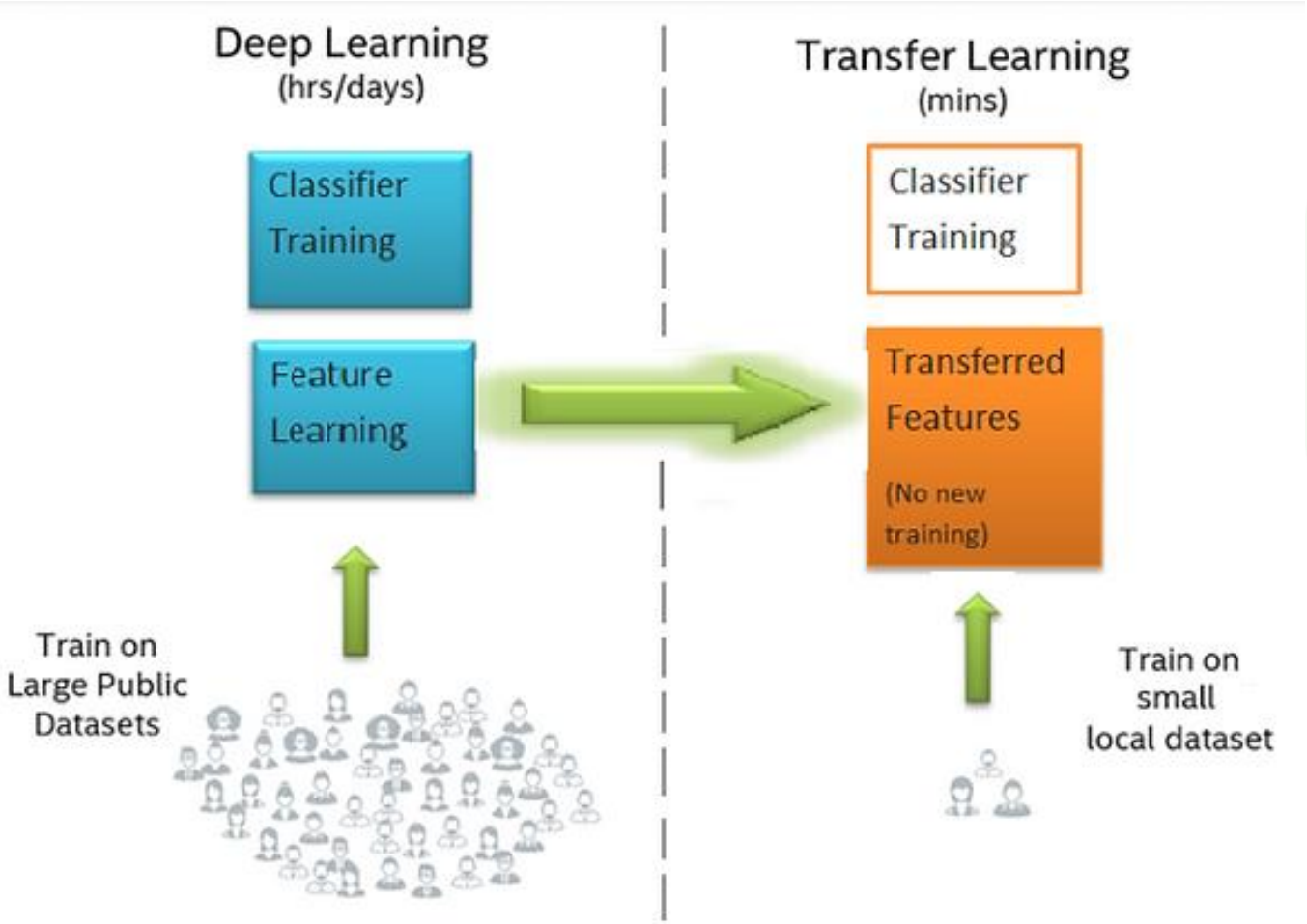
TRANSFER LEARNING

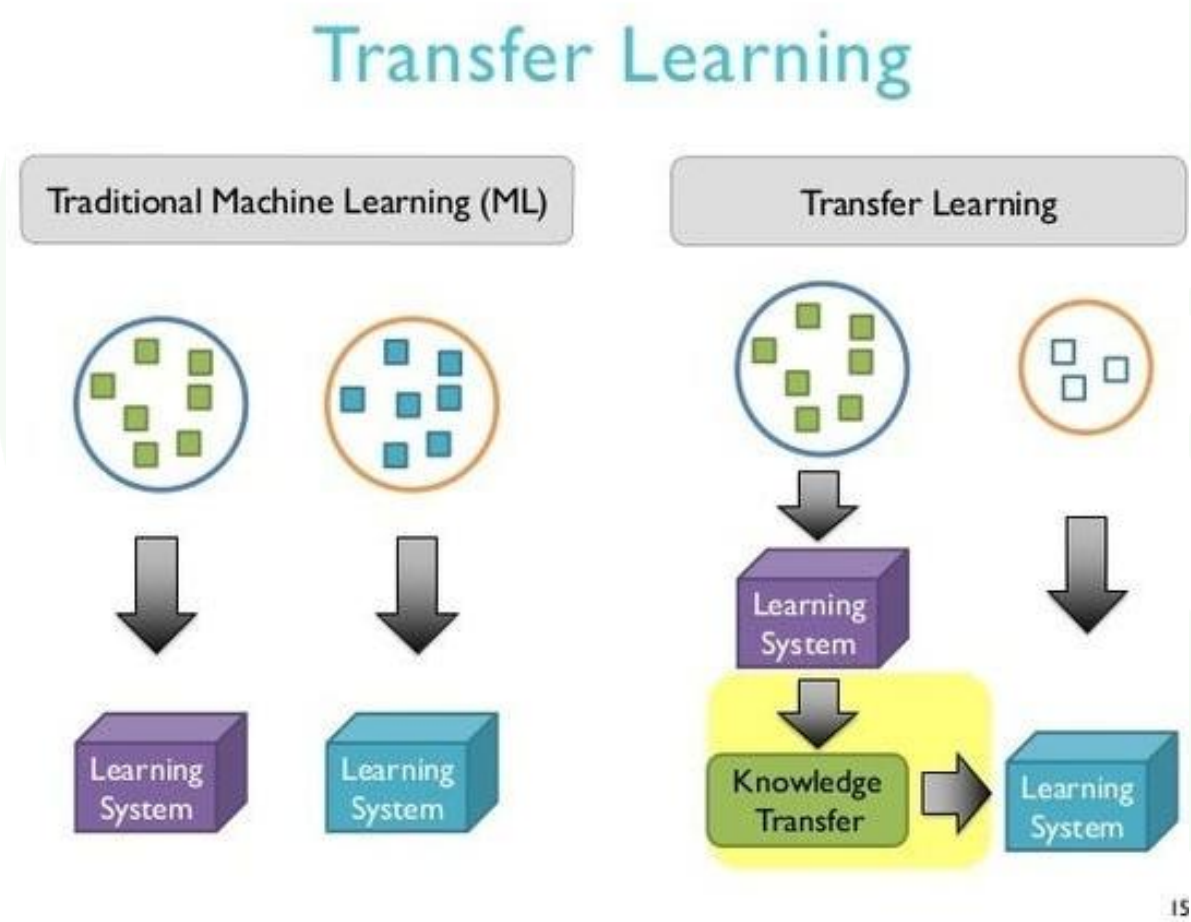
Fine Tuning

FINE TUNING

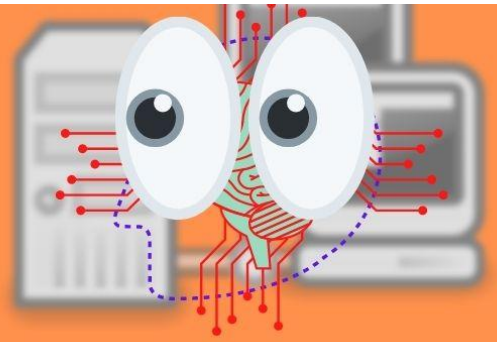


TRANSFER LEARNING



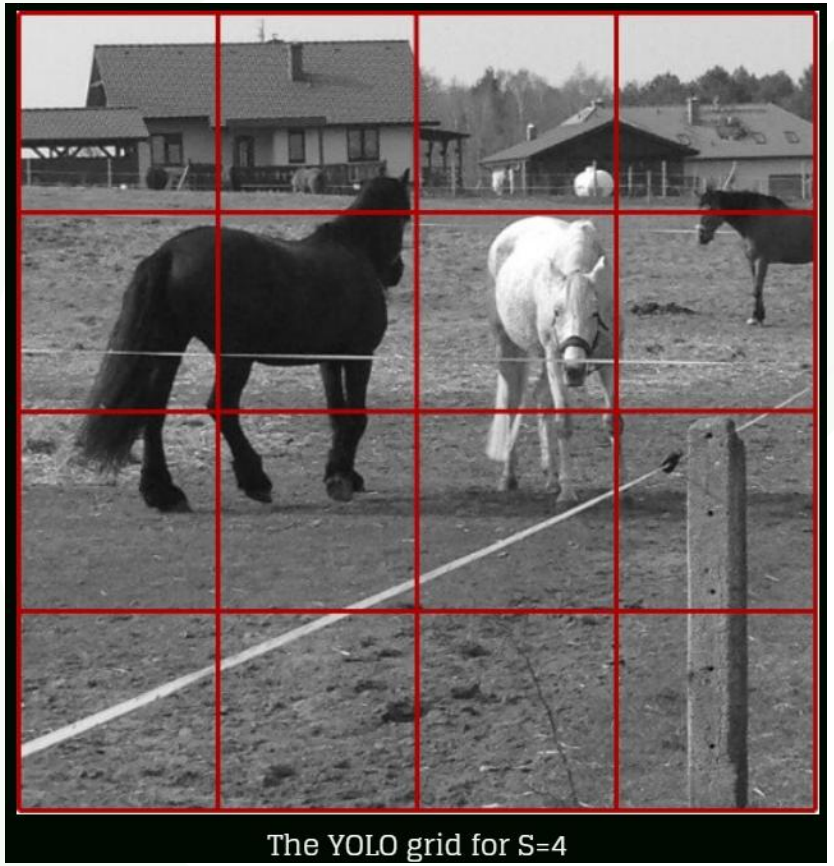


TRANSFER LEARNING



Transfer Learning

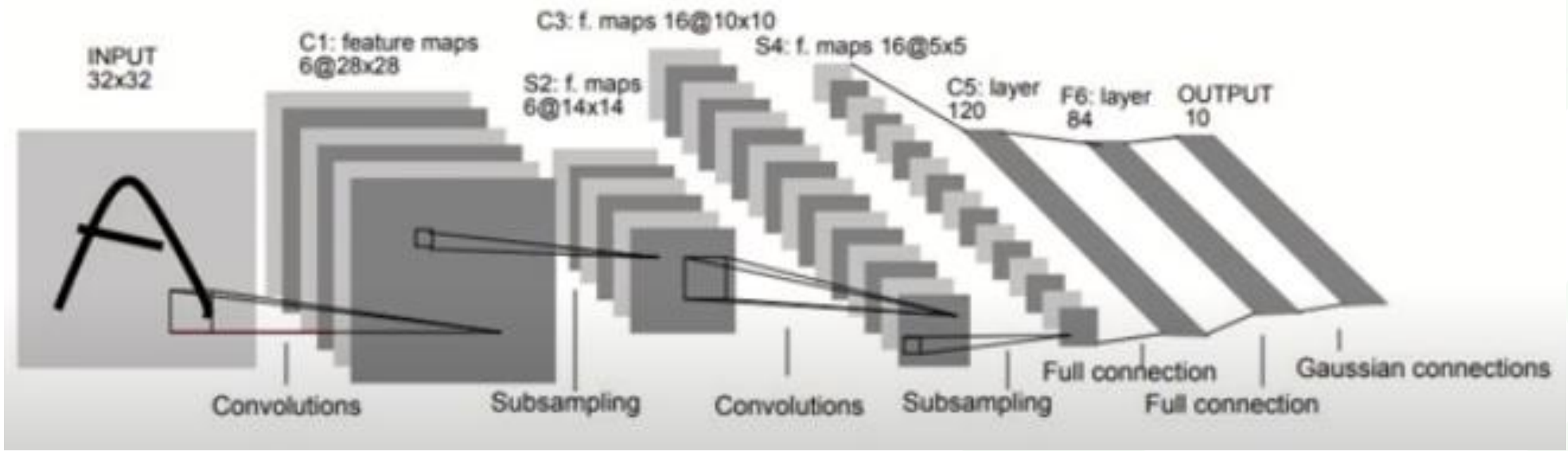
YOLO


The YOLO grid for S=4

https://www.researchgate.net/publication/346516990_Real-time_video_firesmoke_detection_based_on_CNN_in_antifire_surveillance_systems

Popular Image Classification Models and Their Features

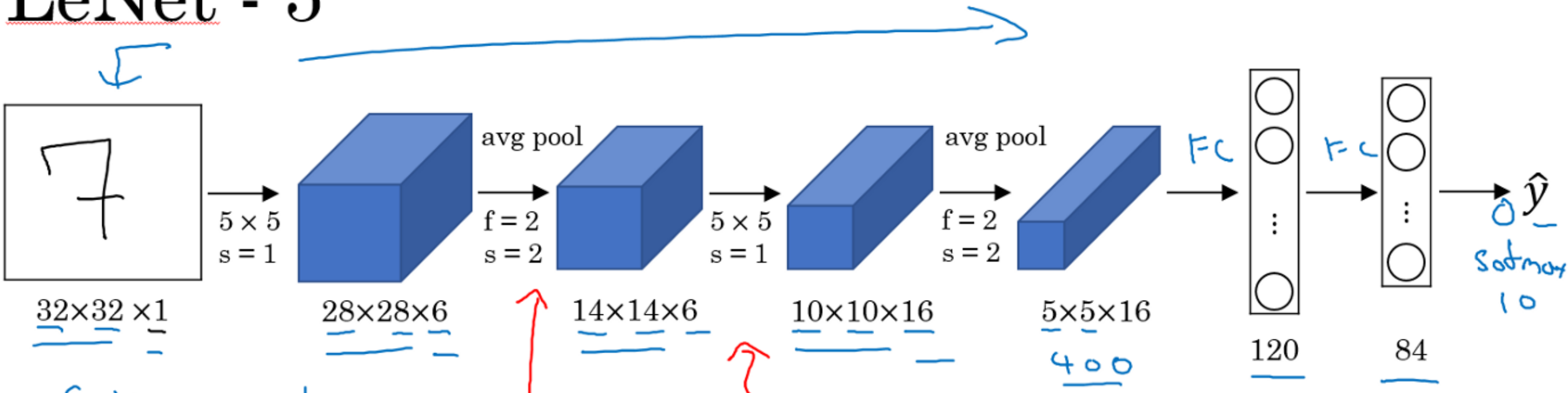
LeNet



LeNet: https://www.youtube.com/watch?v=PiF0I6xif-k

Including model layers

LeNet - 5



60k parameters.

$n_H, n_w \downarrow$ $n_c \uparrow$

Non-linearity after pooling

$n_H \times n_w \times n_c$ $f \times f \times n_c$

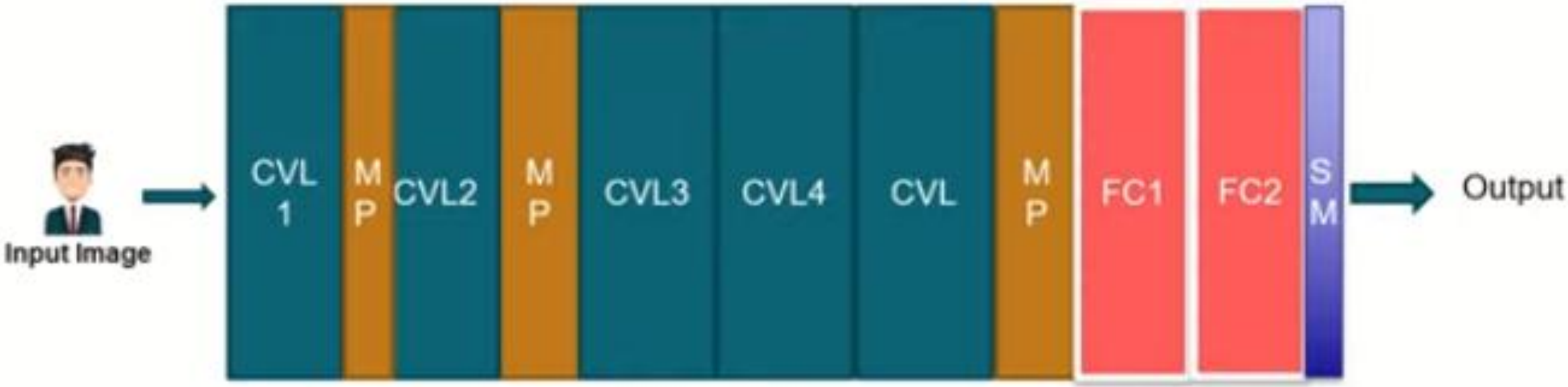
conv pool conv pool fc fc output

Advanced: sigmoid/tanh ReLU

II III

[LeCun et al., 1998. Gradient-based learning applied to document recognition]

AlexNet

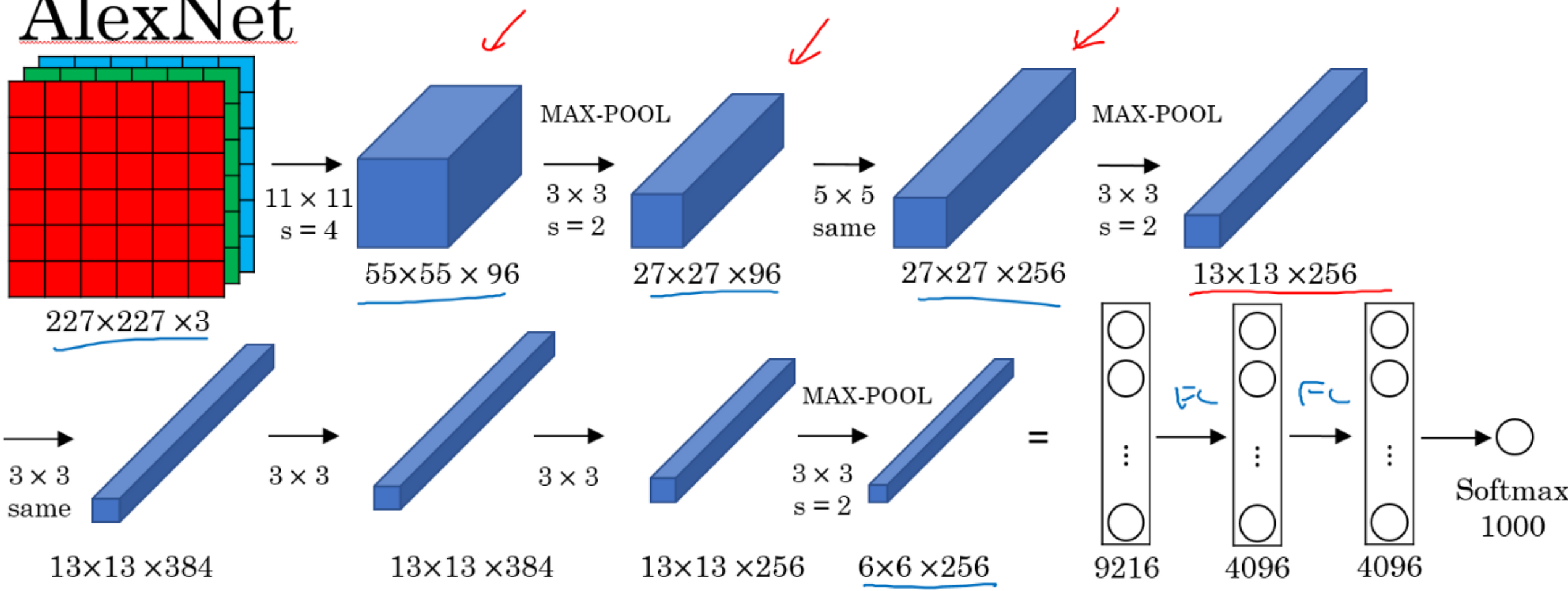


CVL = Convolution Layer
MP = Max Pooling
FC = Fully Connected
SM = Softmax Layer

Including model layers

For AlexNet:
https://www.youtube.com/watch?v=8GheVe2UmUM

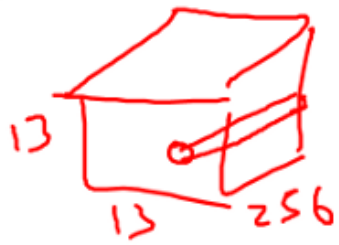
AlexNet



- Similar to LeNet, but much bigger.
- ReLU
- Multiple GPUs.
- Local Response Normalization (LRN)

9216

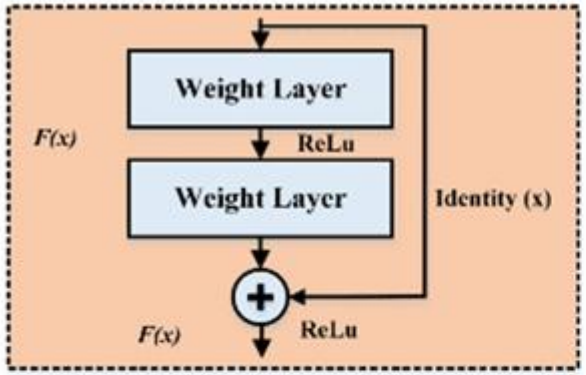
60M parameters



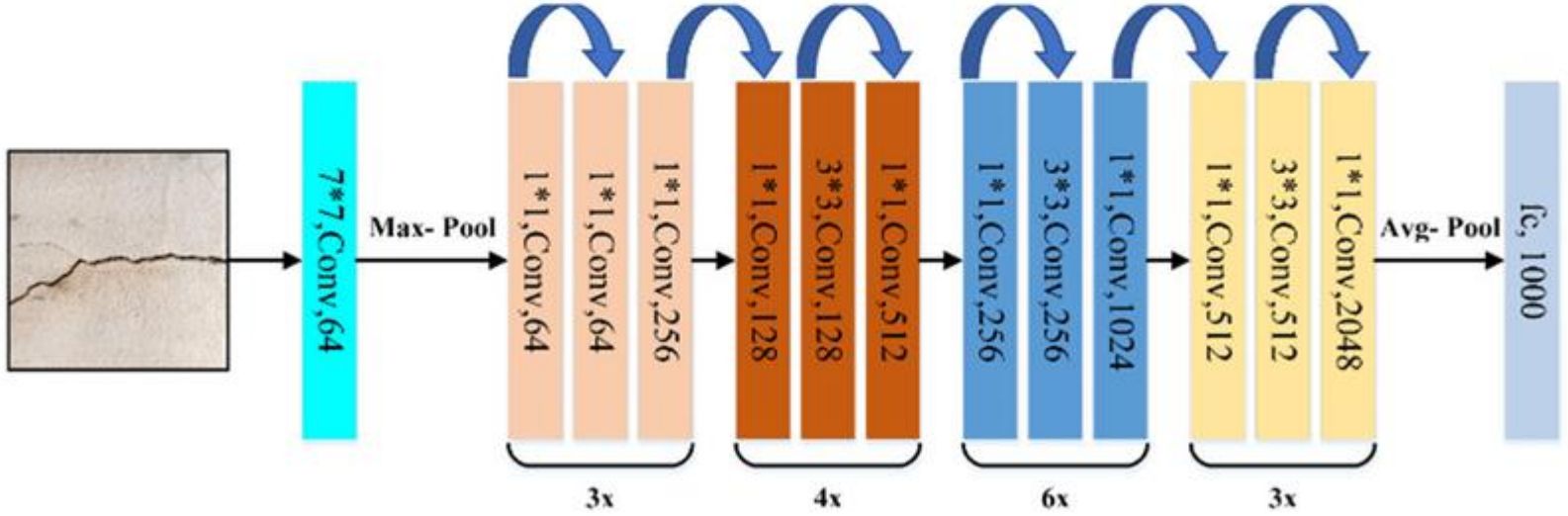
[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Andrew Ng

ResNet50

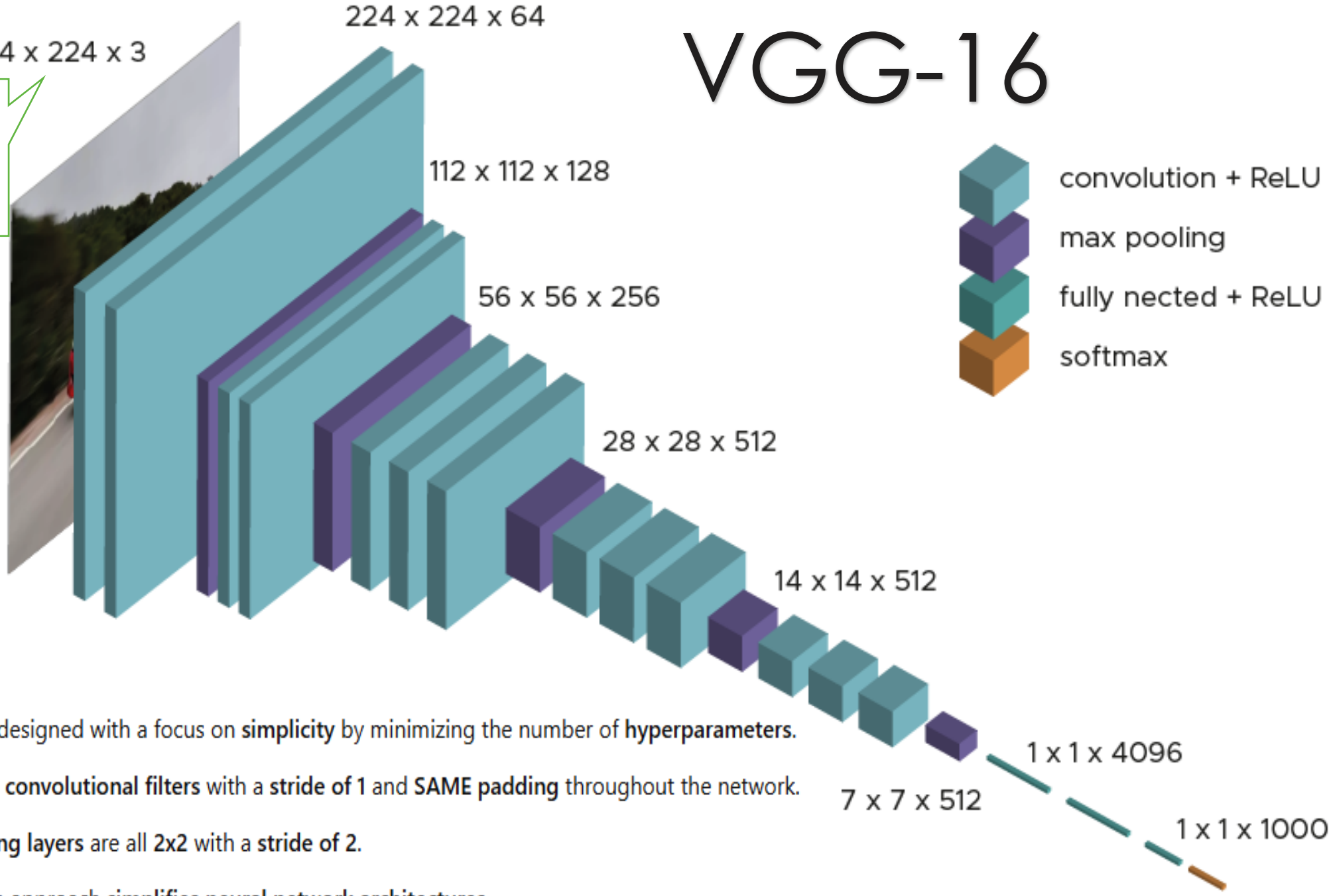


Residual Learning Block



EfficientNet

VGG-16



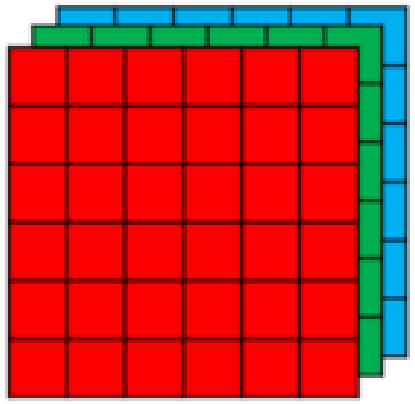
We must consider these sizes

- **VGG-16** is designed with a focus on **simplicity** by minimizing the number of **hyperparameters**.
- It uses **3x3 convolutional filters** with a **stride of 1** and **SAME padding** throughout the network.
- **Max pooling layers** are all **2x2** with a **stride of 2**.
- This design approach **simplifies neural network architectures**.

VGG - 16

VGG-16

CONV = 3×3 filter, s = 1, same MAX-POOL = 2×2 , s = 2



224×224 ×3

→ [CONV 64] ×2 → 224×224×64 → POOL → 112×112 ×64 → [CONV 128] ×2 → 112×112 ×128 → POOL → 56×56 ×128

→ [CONV 256] ×3 → 56×56 ×256 → POOL → 28×28 ×256 → [CONV 512] × 3 → 28×28 ×512 → POOL → 14×14×512

→ [CONV 512] ×3 → 14×14 ×512 → POOL → 7×7×512 → FC 4096 → FC 4096 → Softmax 1000

$n_H, n_W \downarrow$ $n_c \uparrow$

~138M

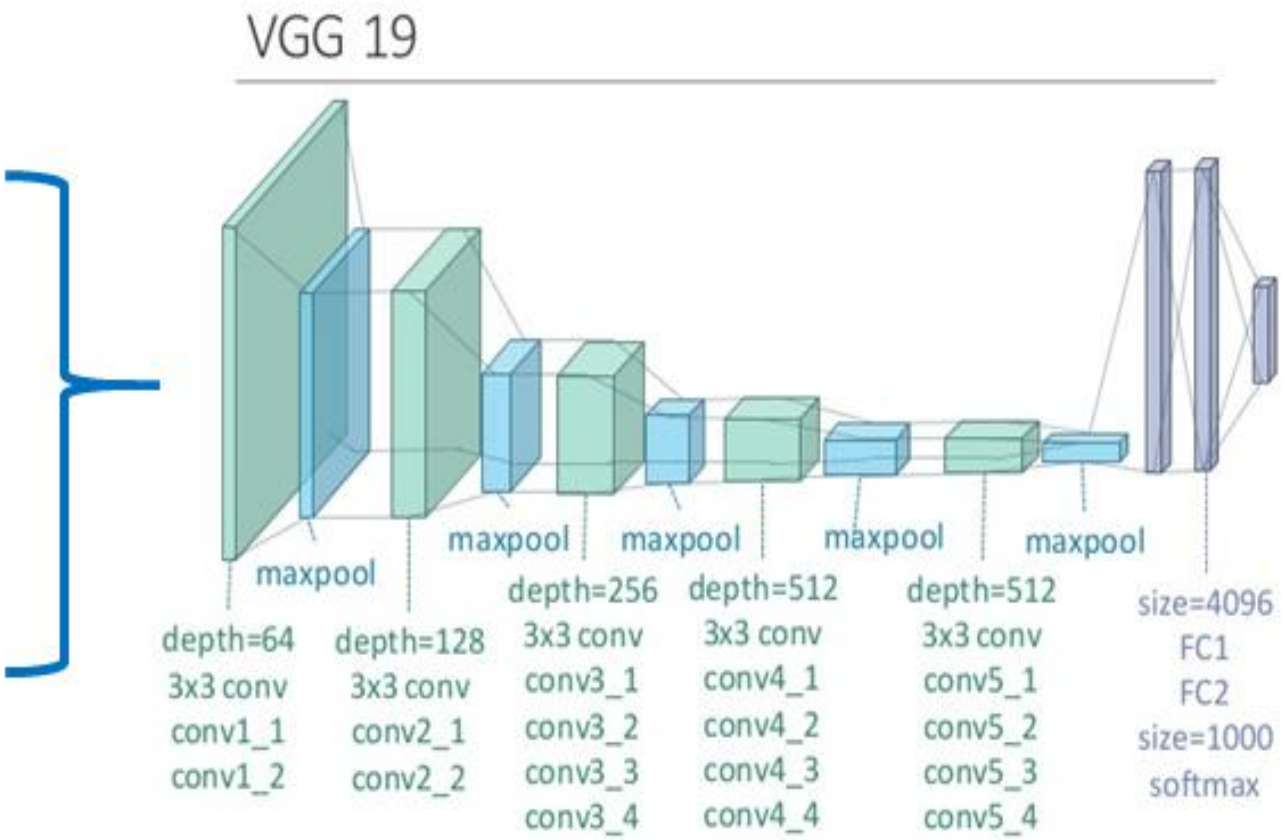
[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

Andrew Ng

VGG-16 and VGG-19

Barcelona city

Impressionist style painting



Barcelona painting with impressionist style

VGG-16 and VGG-19

```
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing.image import load_img, img_to_array

import os
```

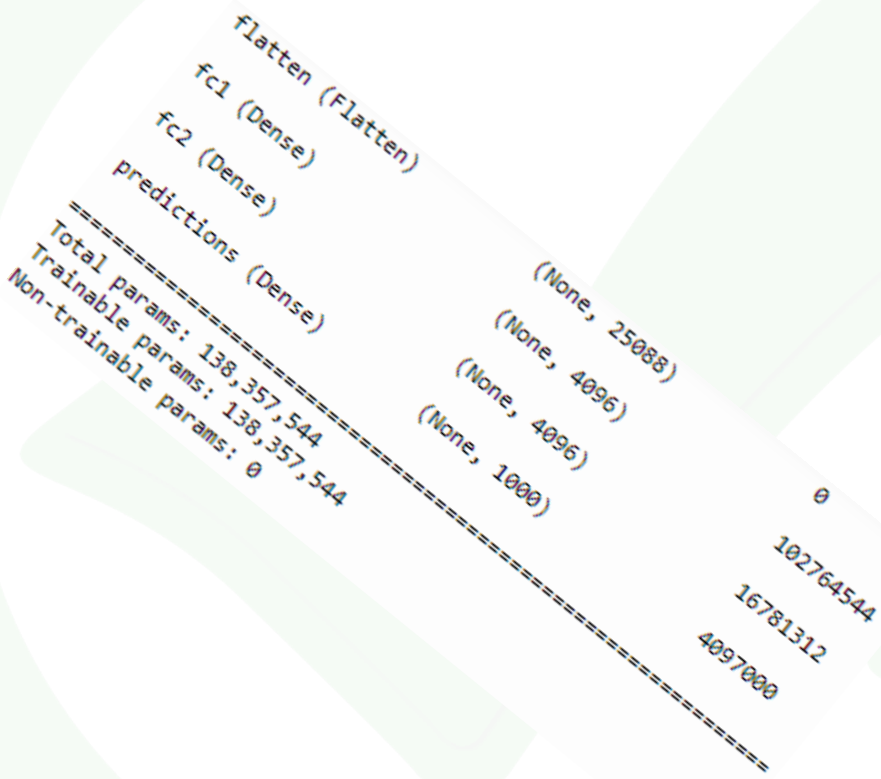
```
model = VGG16()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels.h5
553467904/553467096 [==============================] - 81s 0us/step
553476096/553467096 [==============================] - 81s 0us/step
```

VGG-16 and VGG-19

```
model.summary()
```

```
Model: "vgg16"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 input_1 (InputLayer)         [(None, 224, 224, 3)]     0

 block1_conv1 (Conv2D)        (None, 224, 224, 64)      1792

 block1_conv2 (Conv2D)        (None, 224, 224, 64)      36928

 block1_pool (MaxPooling2D)   (None, 112, 112, 64)      0

 block2_conv1 (Conv2D)        (None, 112, 112, 128)     73856

 block2_conv2 (Conv2D)        (None, 112, 112, 128)     147584

 block2_pool (MaxPooling2D)   (None, 56, 56, 128)       0

 block3_conv1 (Conv2D)        (None, 56, 56, 256)       295168

 block3_conv2 (Conv2D)        (None, 56, 56, 256)       590080

 block3_conv3 (Conv2D)        (None, 56, 56, 256)       590080

 block3_pool (MaxPooling2D)   (None, 28, 28, 256)       0

 block4_conv1 (Conv2D)        (None, 28, 28, 512)       1180160

 block4_conv2 (Conv2D)        (None, 28, 28, 512)       2359808
```



VGG-16 and VGG-19

```
for file in os.listdir('sample'):
    print(file)
```

```
bottle.jpeg
coffee.jpeg
cute_baby.jpeg
```

```
for file in os.listdir('sample'):
    print(file)
    full_path = 'sample/' + file

    image = load_img(full_path, target_size=(224, 224))
    image = img_to_array(image)
    image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
    image = preprocess_input(image)
    y_pred = model.predict(image)
    label = decode_predictions(y_pred, top = 2)
    print(label)
    print()
```

```
bottle.jpeg
[[('n04579432', 'whistle', 0.19901143), ('n03983396', 'pop_bottle', 0.12411169)]]

coffee.jpeg
[[('n07920052', 'espresso', 0.87100804), ('n07930864', 'cup', 0.05682909)]]

cute_baby.jpeg
[[('n03404251', 'fur_coat', 0.44272116), ('n03877472', 'pajama', 0.13531971)]]
```

```
)

model.trainable = False  # Freeze the outer model

assert inner_model.trainable == False  # All layers in `model` are now frozen
assert inner_model.layers[0].trainable == False  # `trainable` is propagated recursively
```

The typical transfer-learning workflow

This leads us to how a typical transfer learning workflow can be implemented in Keras:

1. Instantiate a base model and load pre-trained weights into it.
2. Freeze all layers in the base model by setting `trainable = False`.
3. Create a new model on top of the output of one (or several) layers from the base model.
4. Train your new model on your new dataset.

Note that an alternative, more lightweight workflow could also be:

1. Instantiate a base model and load pre-trained weights into it.
2. Run your new dataset through it and record the output of one (or several) layers from the base model. This is called **feature extraction**.
3. Use that output as input data for a new, smaller model.

A key advantage of that second workflow is that you only run the base model once on your data, rather than once per epoch of training. So it's a lot faster & cheaper.

An issue with that second workflow, though, is that it doesn't allow you to dynamically modify the input data of your new model during training, which is required when doing data augmentation, for instance. Transfer learning is typically used for tasks when your new dataset has too little data to train a full-scale model from scratch, and in such scenarios data augmentation is very important. So in

Summary

- Transfer Learning models were trained on ImageNet.
- So, they are suitable for image classification analyses.
- We can use their architecture and weights (information) by connecting (tuning) our data.
- We should check their image size and is "**pre-processing**" default or not!!!!!!!!!!!!

Deep Learning'de Transfer Learning Modellerini Kullanırken Dikkat Edilmesi Gereken Noktalar

https://medium.com/@ismetgocer09